Codefest 4: Solutions

Hardware for Artificial Intelligence and Machine Learning (HW4AI)

ECE 410/510
Spring 2026

CMAN solutions — INT8 symmetric quantization

Task 1: Scale factor

```
max(|W|) = 2.31   (element W[2][3])
S = 2.31 / 127 = 0.018189...  ≈  0.01819
```

Task 2: INT8 quantized matrix W_q

Apply W_q[i][j] = clamp(round(W[i][j] / S), −128, 127):

```
W_q = [  47,  -66,   19,  115 ]
      [  -4,   50, -103,    7 ]
      [  85,    2,  -24, -127 ]
      [ -10,   57,   42,   30 ]
```

Selected calculations:

```
W[0][0] =  0.85 / 0.01819 =  46.73  -> round = 47
W[0][1] = -1.20 / 0.01819 = -65.97  -> round = -66
W[0][3] =  2.10 / 0.01819 = 115.45  -> round = 115
W[2][3] = -2.31 / 0.01819 =-127.00  -> round = -127
```

Task 3: Dequantized matrix W_deq

Apply W_deq[i][j] = W_q[i][j] × S (S = 0.01819):

```
W_deq = [  0.8549,  -1.2005,   0.3456,   2.0919 ]
        [ -0.0728,   0.9096,  -1.8736,   0.1273 ]
        [  1.5462,   0.0364,  -0.4366,  -2.3101 ]
        [ -0.1819,   1.0369,   0.7640,   0.5458 ]
```

Task 4: Error analysis

```
|err| ≈ [ 0.0049, 0.0005, 0.0056, 0.0081 ]
        [ 0.0028, 0.0004, 0.0064, 0.0073 ]
        [ 0.0038, 0.0064, 0.0034, 0.0001 ]
        [ 0.0019, 0.0069, 0.0060, 0.0042 ]
```

Largest error element: Accept any element with |err| ≥ 0.006. Students must show their calculation.
MAE: ≈0.0042 (accept ±0.001 given rounding variance across calculators).

Task 5: Bad scale experiment

```
W[0][3] = 2.10 / 0.01 = 210  -> clamp to 127  -> error = 0.83
W[2][3] =-2.31 / 0.01 =-231  -> clamp to -128 -> error = 1.03
MAE with S_bad >> MAE with correct S  (~0.15 vs ~0.004)
```

Accept any accurate one-sentence explanation, e.g.: *"When S is too small, large-magnitude weights exceed the INT8 range and are clamped, causing dequantization errors proportional to how far the true value exceeds 127 × S."*

CLLM solutions — MAC unit review and correction

Reference correct implementation: mac_correct.v

```
module mac (
    input  logic        clk,
```

```
    input  logic        rst,
    input  logic signed [7:0]  a,
    input  logic signed [7:0]  b,
    output logic signed [31:0] out
);
    logic signed [15:0] product;
    assign product = a * b;   // 8x8 signed -> 16-bit signed

    always_ff @(posedge clk) begin
        if (rst)
            out <= 32'sd0;
        else
            out <= out + 32'(product);  // sign-extend to 32-bit
    end
endmodule
```

Expected simulation outputs

```
Cycle 1: a= 3, b=4  -> out =  12
Cycle 2: a= 3, b=4  -> out =  24
Cycle 3: a= 3, b=4  -> out =  36
Reset:   rst=1      -> out =   0
Cycle 5: a=-5, b=2  -> out = -10
Cycle 6: a=-5, b=2  -> out = -20
```

Common issues: grading reference

Issue	Typical LLM code	Correct version
Non-synthesizable always block	always @(posedge clk or posedge rst)	always_ff @(posedge clk) with if (rst) inside
Unsigned multiply	out <= out + a * b; // unsigned result	Declare signed [15:0] product; assign product = a * b; use 32'(product)
Output not signed	output reg [31:0] out	output logic signed [31:0] out
initial block for reset	initial begin out = 0; end	Remove; use always_ff with rst signal
No overflow note	(no comment on 32-bit wrap)	Document wrap-around; note saturation would need extra logic

Grading notes

- Accept any technically correct identification of a synthesizability, correctness, or style problem, not only those listed above.
- The corrected version need not match the reference exactly. Accept any version that compiles, simulates correctly, and avoids the identified issue.
- Simulation evidence: accept a terminal log, pytest output, GTKWave screenshot, or CI badge. The log must show: 12, 24, 36, 0, −10, −20.

COPT solutions — cocotb and project core

Expected cocotb output (passing)

```
     0.00ns INFO     Running tests with cocotb v1.9.x
   100.00ns INFO     test_mac.test_mac_basic      PASSED
   200.00ns INFO     test_mac.test_mac_overflow   PASSED
     0.00ns INFO     Results: 2 passed, 0 failed
```

Overflow behavior

The reference design wraps on overflow (two's complement). Both wrap and saturation are acceptable as long as the behavior is correctly documented.

Part B evaluation guidance

- The project HDL module need not be functionally complete. It must have correct port declarations, use always_ff for sequential logic, and simulate without compile errors.
- The testbench stub must instantiate the module and drive at least one clock cycle. Passing assertions are not required.
- The interface justification paragraph should reference the student's M1 arithmetic intensity. If M1 is not yet complete, accept a placeholder with a note that it will be updated.